Pontificia Universidad Javeriana
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas

Sistemas Operativos

Taller de Evaluación de Rendimiento

Multiplicación de matrices con hilos POSIX y procesos `fork` en tres sistemas de cómputo

Integrantes

Santiago Arias Tovar

Juan Sebastian Quintero Gil

Jesus Daniel Duque Aparicio

Pablo Samuel Leon Hernandez

Katheryn Camacho Gonzalez

Profesor

John Corredor

Bogotá, D. C.
Octubre de 2026

Índice

1. Introducción **2**

2. Métricas de desempeño **2**

3. Plataformas de hardware y software **2**
3.1. Condiciones de ejecución 4

4. Programas evaluados y automatización **4**
4.1. Algoritmos 4
4.2. Automatización con `lanzador.pl` 5

5. Diseño de experimentos **5**
5.1. Factores y niveles 5
5.2. Tamaños de matriz y jerarquía de memoria 5
5.3. Procedimiento 7
5.4. Calidad de los datos 7

6. Análisis de resultados **7**
6.1. Tiempo en serie (1 hilo) 7
6.2. Algoritmo: FxC frente a FxT 7
6.3. Hilos POSIX frente a procesos fork 9
6.4. Comparación entre el Ryzen, el Core 2 Duo y el HP Victus 9
6.4.1. Tiempo en serie (1 hilo) 11

7. Evaluación en Entorno Virtualizado (Intel Core i5-13400F) **11**

8. Limitaciones **12**

9. Conclusiones y recomendaciones **12**
9.1. Conclusiones 12
9.2. Recomendaciones 13

A. Comandos para repetir el experimento **13**

1. Introducción

El rendimiento de un programa no depende solo del algoritmo: depende de cómo ese algoritmo usa la jerarquía de memoria, de cuántas unidades de cálculo puede aprovechar y de los costos que le impone el sistema operativo al crear hilos o procesos. Este taller estudia esas tres influencias con un problema clásico, la multiplicación de matrices cuadradas $C = A \times B$ de $N \times N$ elementos de tipo `double`, cuyo costo es $O(N^3)$.

Se comparan:

- dos algoritmos: el clásico *fila por columna* (FxC) y el de *fila por transpuesta* (FxT), que recorre las dos matrices por filas;
- dos mecanismos de concurrencia en C: hilos POSIX (`pthread`) y procesos creados con `fork`;
- la versión serie (1 hilo) y la paralela (2, 4, 6, 8, 12 y 16 hilos o procesos);
- tres sistemas de cómputo con Linux: una laptop con un Ryzen 5 5600H, un servidor doméstico con un Core 2 Duo E4500 y una laptop HP Victus con un Core i5-12450H y Linux Mint.

El informe describe las métricas de desempeño usadas, las plataformas de hardware y software, el diseño de la batería de experimentos, el análisis de los resultados y las conclusiones y recomendaciones que se desprenden de ellos. Cada configuración se midió 30 veces para que el promedio sea representativo (ley de los grandes números).

2. Métricas de desempeño

Las métricas de uso común para evaluar programas, secuenciales o paralelos, son las siguientes [1, 2].

Tiempo de ejecución (T). Tiempo transcurrido entre el inicio y el fin del cálculo. Es la métrica más directa; menor es mejor. Para un programa paralelo con p hilos se denota T_p, y T_1 es el tiempo en serie.

Tiempo de respuesta frente a tiempo de CPU. El primero incluye esperas y planificación del sistema operativo; el segundo solo el tiempo en que el programa ejecuta en CPU. En paralelo, el tiempo de CPU suma el de todos los hilos y crece aunque el tiempo de pared baje.

Métricas usadas en el taller. La métrica principal es el **tiempo de ejecución de pared**, medido por el propio programa en microsegundos y promediado sobre 30 repeticiones.

3. Plataformas de hardware y software

Las pruebas se hicieron de forma nativa (sin máquinas virtuales) en tres sistemas con Linux. La Tabla 1 resume sus características.

Software común. Los tres sistemas compilaron las mismas fuentes con el mismo `Makefile` (`make All`, que invoca `gcc -O3`) y ejecutaron el mismo `lanzador.pl`. Los compiladores difieren en versión, lo que puede producir código ligeramente distinto; se señala en las limitaciones.

Tabla 1: Plataformas de hardware y software utilizadas.

	A: laptop Dell G15 5515	**B: Core 2 DUO E4500**	**C: laptop HP Victus**
Procesador	AMD Ryzen 5 5600H (Zen 3, 7 nm, 2021)	Intel Core 2 Duo E4500 (65 nm, 2007)	Intel Core i5-12450H (Alder Lake, Intel 7, 2022)
Núcleos / CPUs lógicas	6 / 12 (SMT)	2 / 2 (sin Hyper-Threading)	8 / 12 (4 núcleos P con Hyper-Threading y 4 núcleos E)
Frecuencia	3.3 GHz base, hasta 4.28 GHz	1.2–2.2 GHz	Núcleos P: 2.0 GHz base, hasta 4.4 GHz. Núcleos E: hasta 3.3 GHz
Caché L1 de datos	32 KiB por núcleo	32 KiB por núcleo	48 KiB por núcleo P, 32 KiB por núcleo E
Caché L2	512 KiB por núcleo	2 MiB compartida	1.25 MiB por núcleo P, 2 MiB por grupo de 4 núcleos E
Caché L3	16 MiB compartida	No tiene	12 MiB compartida
RAM	14 GiB utilizables	2 GB (1.9 GiB utilizables)	16 GB DDR4-3200 (15 GiB utilizables), doble canal
Acceso a memoria	Controlador integrado	Bus FSB 800 MT/s (≈6.4 GB/s)	Controlador integrado (≈51 GB/s teóricos)
Sistema operativo	Debian 13.7, kernel 6.12, con escritorio	Debian 12, kernel 6.1, sin escritorio	Linux Mint 22, kernel 6.8, con escritorio (Cinnamon)
Compilador	gcc 14.2, `-O3`	gcc 12.2, `-O3`	gcc 13.3, `-O3`
Instalación	Nativa	Nativa	Nativa

3.1. Condiciones de ejecución

En el Ryzen y el Core 2 Duo se detuvo todo servicio que pudiera interferir (Pi-hole, Tailscale, cron, sincronización de reloj, Bluetooth, impresión y Steam), se fijó el gobernador de frecuencia en `performance`, y no se usó el equipo mientras corría la batería. Los lanzamientos se hicieron con `nohup` para sobrevivir al cierre de sesión. En el HP Victus se usó el mismo protocolo, con el equipo conectado a la corriente, el perfil de energía en rendimiento y una base de refrigeración.

Tabla 2: Condiciones observadas durante la batería.

	Ryzen 5 5600H	**Core 2 Duo E4500**	**HP Victus**
Duración de la batería	≈ 2 h 30 min	≈ 17 h	≈ 2 h 50 min
Suma de los tiempos medidos	2.4 h	16.8 h	2.5 h
Temperatura máxima	92.6 °C (ventiladores al máximo)	No se registró	87.3 °C (ventiladores al máximo)
Frecuencia bajo carga	3.2–4.0 GHz	2.2 GHz fija	Núcleos P 3.6–4.2 GHz, núcleos E 2.8–3.3 GHz
Estrangulamiento térmico	No observado	No aplica	Leve, por límite de potencia en los primeros minutos

4. Programas evaluados y automatización

4.1. Algoritmos

Se evaluaron cuatro ejecutables: `mxmPosixFxC`, `mxmPosixFxT`, `mxmForkFxC` y `mxmForkFxT`. Reciben dos argumentos, el tamaño de matriz N y el número de hilos o procesos, e imprimen el tiempo de ejecución en microsegundos.

- **FxC (fila por columna).** Calcula $C_{ij} = \sum_k A_{ik}B_{kj}$ leyendo A por filas y B por columnas. Cada paso por una columna de B salta $8N$ bytes, así que cada lectura cae en una línea de caché distinta y el desempeño depende de cuánto cabe en caché.
- **FxT (fila por transpuesta).** Usa la transpuesta de B para que ambas lecturas sean contiguas. La aritmética es la misma, pero el acceso a memoria es secuencial y favorece a la caché y al *prefetcher*.
- **Paralelización.** El trabajo se reparte por bandas de filas de C entre los p hilos o procesos, de modo que N debe ser divisible entre p. Esto condiciona el diseño de la batería (Sección 5).
- **Hilos POSIX frente a `fork`.** Los hilos comparten el espacio de direcciones. Los procesos hijos heredan una copia (copy-on-write) y su creación es más cara. En las versiones `fork`, `matC` no es memoria compartida, así que el proceso padre no ve el resultado calculated por los hijos. El tiempo medido es válido para evaluar rendimiento, pero estas versiones no devuelven una matriz utilizable; se debe tener presente al interpretar los resultados.

4.2. Automatización con `lanzador.pl`

El script en Perl recorre la batería completa y guarda cada configuración en un archivo de la carpeta `Soluciones` con nombre de la forma

`<programa>-<N>-Hilos-<p>.dat`

con una línea por repetición (tiempo en microsegundos). Sus parámetros se definen así:

```
@Nombre_Ejecutable = ("mxmPosixFxC","mxmPosixFxT","mxmForkFxC","mxmForkFxT");
@Size_Matriz = ("96","240","480","720","960","1440","1920","2400");
@Num_Hilos = (1,2,4,6,8,12,16);
$Repeticiones = 30;
...
system("\"$Path/$Binarios/$nombre\" $size $hilo >> \"$file\"");
```

Respecto al script original se hicieron cuatro cambios:

1. Se cambiaron las listas de tamaños e hilos y se subieron las repeticiones de 4 a 30.
2. Se activó la línea `system(...)`, que venía comentada, de modo que el script ejecuta los programas y no solo imprime los comandos.
3. Se agregaron comillas a las rutas, pues sin ellas el script falla si la carpeta tiene espacios.
4. Se eliminó el `printf` que imprimía cada comando y llenaba el registro de miles de líneas.

El script define los cuatro ejecutables, los ocho tamaños, los siete números de hilos y las 30 repeticiones. Para cada combinación arma la ruta del binario (`$Path/$Binarios/$nombre`), lo invoca con `system` pasando el tamaño y el número de hilos, y redirige la salida al archivo de la configuración, de modo que cada archivo acumula las 30 mediciones. Las comillas en las rutas evitan errores cuando hay espacios.

5. Diseño de experimentos

5.1. Factores y niveles

Se ejecutaron los cuatro programas con todas las combinaciones de los factores de la Tabla 3.

El experimento es un diseño factorial completo: 4 programas $\times$ 8 tamaños $\times$ 7 números de hilos $= 224$ configuraciones, cada una con 30 repeticiones, es decir, **6 720 ejecuciones por máquina** y **20 160 en total**. Un mismo conjunto de niveles en las tres máquinas hace los datos directamente comparables.

5.2. Tamaños de matriz y jerarquía de memoria

Con tres matrices de `double`, el conjunto de trabajo es de $24N^2$ bytes. La Tabla 4 indica en qué nivel de la jerarquía cabe en cada máquina.

La memoria necesaria ($\leq$ 138 MB) cabe sin problema en la RAM de las tres máquinas, por lo que no hay intercambio a disco.

Tabla 3: Factores del experimento y justificación de sus niveles.

Factor	**Niveles**	**Justificación**
Tamaño N	96, 240, 480, 720, 960, 1440, 1920, 2400	Todos son múltiplos de 48, así que se dividen exactamente entre cada número de hilos (requisito del enunciado). Recorren la jerarquía de memoria: desde matrices que caben en L2 hasta otras que obligan a usar la RAM.
Hilos o procesos p	1, 2, 4, 6, 8, 12, 16	1 es la versión serie. 2 son los núcleos del Core 2 Duo. 4 son los núcleos P del HP Victus. 4 y 8 son valores del enunciado. 6 son los núcleos del Ryzen. 12 son las CPUs lógicas del Ryzen y del HP Victus. 16 supera el número de CPUs de las tres máquinas y permite observar la sobresuscripción.
Algoritmo	FxC, FxT	Contrastar el acceso a memoria.
Mecanismo	Posix, Fork	Contrastar el costo de crear hilos y procesos.
Repeticiones	30 por combinación	Mínimo exigido; reduce el ruido del sistema (Sección 5.4).

Tabla 4: Dónde caben las tres matrices ($24N^2$ bytes) en cada máquina.

N	Memoria	Ryzen 5 5600H	Core 2 Duo E4500	HP Victus
96	0.2 MB	L2	L2	L2
240	1.4 MB	L3	L2 (justo)	L3
480	5.5 MB	L3	RAM	L3
720	12.4 MB	L3 (justo)	RAM	L3 (justo)
960–2400	22–138 MB	RAM	RAM	RAM

5.3. Procedimiento

1. Copiar y descomprimir el `.tar` con las fuentes en cada máquina.
2. Compilar con `make All` (en equipos sin compilador, instalar `build-essential`).
3. Verificar la corrección de los cuatro programas con $N = 4$ y 2 hilos.
4. Calibrar a mano algunos casos grandes (por ejemplo `./bin/mxmPosixFxC 1440 1`) para estimar la duración de la batería.
5. Detener los servicios que pudieran interferir y fijar el gobernador de CPU en `performance`.
6. Lanzar `lanzador.pl` con `nohup` y no usar el equipo mientras corre.
7. Al terminar, comprobar que existan 224 archivos con 30 líneas cada uno (6 720 mediciones).

Los comandos usados para repetir el experimento se recogen en el Anexo A.

5.4. Calidad de los datos

Los datos de las tres máquinas están completos (224 de 224 archivos con 30 mediciones) y son estables (Tabla 5).

Tabla 5: Calidad de los datos.

Indicador	Ryzen 5 5600H	Core 2 Duo E4500	HP Victus
Archivos completos (30 mediciones)	224 / 224	224 / 224	224 / 224
CV (mediana)	2.5 %	3.7 %	3.7 %
Configuraciones con CV mayor a 10 %	33 de 224	21 de 224	26 de 224

Las variaciones grandes (15–67 %) se concentran en $N = 96$ y $N = 240$ con muchos hilos, donde una ejecución dura menos de 4 ms y cualquier interrupción pesa tanto como el cálculo. En el Core 2 Duo también aparecen con 2 hilos y matrices grandes (16–18 %), lo que se discute en la Sección 6. En esos casos conviene apoyarse también en la mediana. Los del HP Victus se discuten en la Sección 6.

6. Análisis de resultados

6.1. Tiempo en serie (1 hilo)

Con FxT el Ryzen es siempre unas 2.5 veces más rápido: es la diferencia de velocidad *pura* de un núcleo. Con FxC la diferencia llega a 7.8× en $N = 480$: en el Ryzen las matrices aun caben en su L3 de 16 MiB, mientras que el Core 2 Duo, que no tiene L3, ya va a la RAM. Desde $N = 960$ ambos van a RAM y la diferencia baja a 3–5×.

6.2. Algoritmo: FxC frente a FxT

Mientras las matrices caben en caché, los dos algoritmos rinden igual (cociente ≈ 1). En cuanto dejan de caber, FxT gana: desde $N = 480$ en el Core 2 Duo (3.2×) y desde $N = 960$ en el Ryzen (1.7×). La ventaja es mayor en la máquina con menos caché. El pico en $N = 1920$ (4.2× y 3.2×) se debe a la anomalía que se explica más abajo.

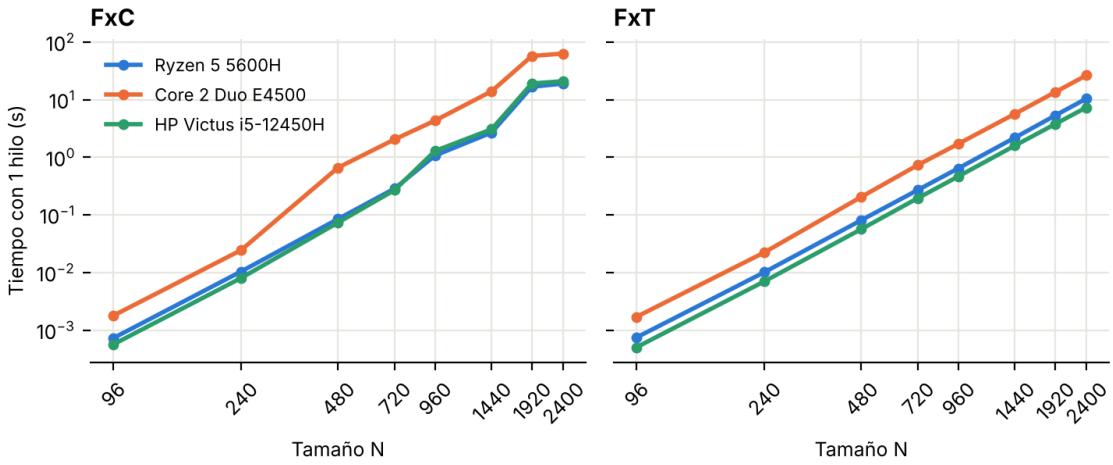


Figura 1: Tiempo con 1 hilo, versiones POSIX. Escala logarítmica.

Tabla 6: Tiempo con 1 hilo, versiones POSIX, y cuántas veces es más rápido el Ryzen.

N	FxC C2D	FxC Ryzen	Ryzen más rápido	FxT C2D	FxT Ryzen	Ryzen más rápido
96	0.002 s	0.001 s	2.5×	0.002 s	0.001 s	2.3×
240	0.024 s	0.010 s	2.3×	0.022 s	0.010 s	2.2×
480	0.660 s	0.085 s	7.8×	0.205 s	0.081 s	2.5×
720	2.045 s	0.287 s	7.1×	0.735 s	0.271 s	2.7×
960	4.316 s	1.068 s	4.0×	1.706 s	0.641 s	2.7×
1440	13.978 s	2.685 s	5.2×	5.667 s	2.187 s	2.6×
1920	57.134 s	16.708 s	3.4×	13.463 s	5.297 s	2.5×
2400	63.136 s	18.951 s	3.3×	26.421 s	10.396 s	2.5×

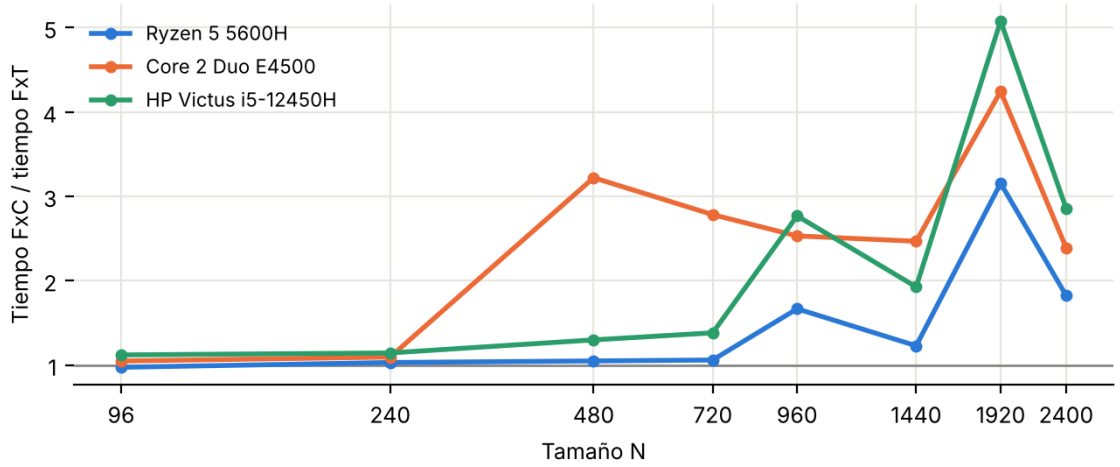


Figura 2: Cociente tiempo FxC / tiempo FxT con 1 hilo. Por encima de 1, FxT es más rápido.

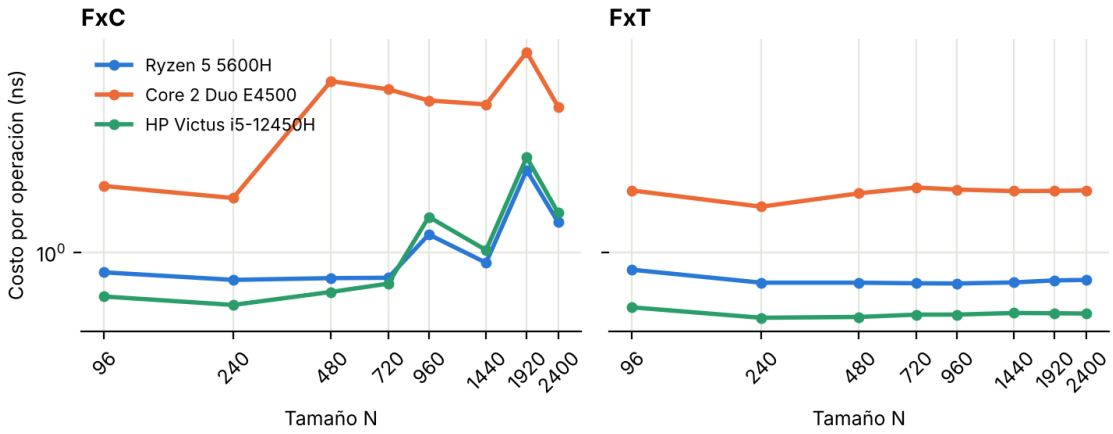


Figura 3: Costo de cada multiplicación-suma (T/N^3) con 1 hilo.

- **FxT tiene un costo plano:** 0.73 ns en el Ryzen , 1.9 ns en el Core 2 Duo, y 0.96 en el HP Victus, sin importar N. Lee las dos filas de forma contigua, así que la caché siempre tiene listo el siguiente dato.
- **FxC depende de la caché:** recorre la columna de B saltando $8N$ bytes y cada lectura cae en una línea distinta.

Anomalía en $N = 1920$. En las tres máquinas FxC cuesta más por operación con $N = 1920$ que con $N = 2400$ (Ryzen: 2.36 contra 1.37 ns; Core 2 Duo: 8.07 contra 4.57 ns). Como $1920 = 15 \times 128$, el salto entre elementos de una columna es múltiplo de una potencia de 2 grande, y todos caen en pocos conjuntos de la caché, que se expulsan entre sí. En la L2 del Ryzen (1 024 conjuntos) la columna solo puede usar 64. Pasa lo mismo, más leve, con $N = 960$ [5]. Que aparezca en dos procesadores tan distintos indica que es un efecto del tamaño y del patrón de acceso, no de una máquina.

6.3. Hilos POSIX frente a procesos fork

- $N = 96$ (menos de 2 ms de cálculo): en el Ryzen, `fork` es entre 20 % y 90 % más lento, porque crear procesos y copiar páginas (copy-on-write) cuesta más que crear hilos. En el Core 2 Duo el resultado es irregular por el ruido (variación de hasta 67 %), al igual que en el HP Victus.

6.4. Comparación entre el Ryzen, el Core 2 Duo y el HP Victus

En el Ryzen gana casi siempre Posix FxT con 12 hilos; en el Core 2 Duo, FxT con 2 hilos o procesos (en 6 de los 8 tamaños). La mejor configuración coincide con el número de CPUs de cada máquina. La ventaja del Ryzen crece con el tamaño (3× en $N = 96$, hasta 15× en N grandes): con más trabajo puede aprovechar sus 12 CPUs, mientras el Core 2 Duo se queda en 2. El rendimiento máximo es de 25.5 GFLOPS en el Ryzen (Posix FxT, $N = 2400$, 12 hilos) contra ≈1.9 GFLOPS en el Core 2 Duo.

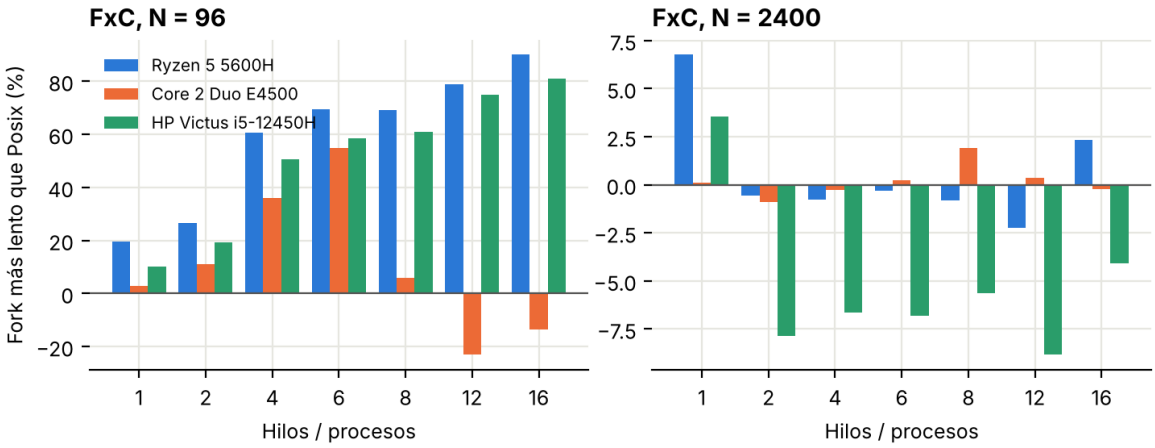


Figura 4: Cuánto más lento es Fork que Posix (FxC) con matrices pequeñas y grandes. Las escalas son distintas.

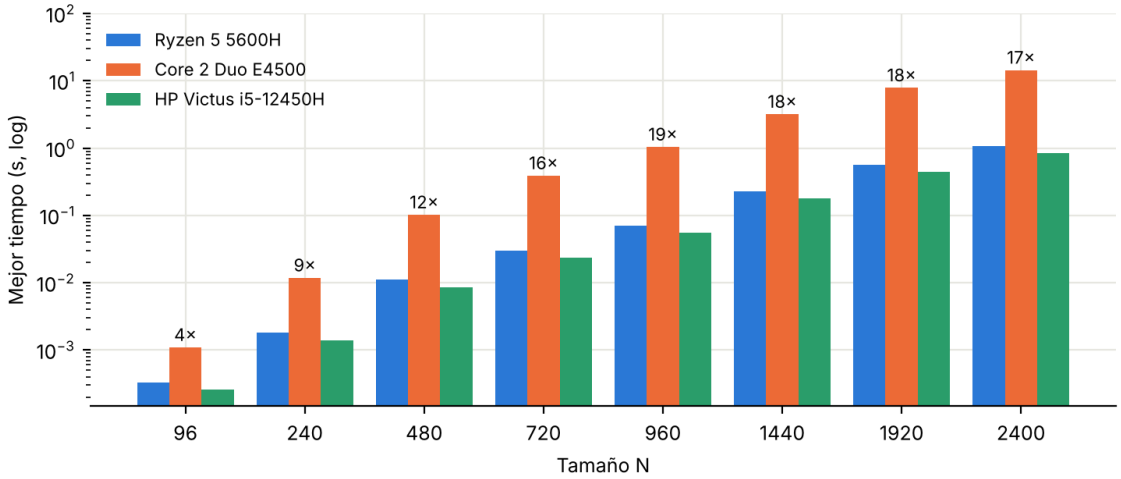


Figura 5: Mejor tiempo de cada máquina por tamaño. Sobre las barras: cuántas veces más lenta fue la peor máquina frente a la mejor.

Tabla 7: Mejor configuración de cada máquina por tamaño.

N	Mejor en Ryzen	Tiempo	Mejor en Core 2 Duo	Tiempo	Ryzen más rápido
96	PosixFxC, 4 hilos	0.33 ms	PosixFxC, 2 hilos	1.09 ms	3.3×
240	PosixFxT, 12 hilos	1.80 ms	PosixFxT, 2 hilos	11.8 ms	6.5×
480	PosixFxT, 12 hilos	11.1 ms	PosixFxT, 6 hilos	102 ms	9.2×
720	PosixFxT, 12 hilos	29.8 ms	PosixFxT, 2 hilos	389 ms	13.0×
960	ForkFxT, 12 hilos	70.3 ms	PosixFxT, 2 hilos	1 039 ms	14.8×
1440	PosixFxT, 12 hilos	229 ms	ForkFxT, 2 hilos	3 182 ms	13.9×
1920	PosixFxT, 12 hilos	557 ms	ForkFxT, 2 hilos	7 867 ms	14.1×
2400	PosixFxT, 12 hilos	1 085 ms	ForkFxT, 2 hilos	14 387 ms	13.3×

6.4.1. Tiempo en serie (1 hilo)

Tabla 8: HP Victus. Tiempo con un hilo (ms), versiones POSIX, y costo por operación T/N^3 (ns).

N	FxC (ms)	FxT (ms)	FxC / FxT	FxT (ns/op)
96	0.56	0.50	1.12	0.57
240	8.01	7.01	1.14	0.51
480	73.4	56.5	1.30	0.51
720	270	195	1.38	0.52
960	1 282	463	2.77	0.52
1440	3 064	1 591	1.93	0.53
1920	19 091	3 760	5.08	0.53
2400	20 904	7 322	2.86	0.53

7. Evaluación en Entorno Virtualizado (Intel Core i5-13400F)

Como experimento complementario, se ejecutó la misma batería de pruebas en un entorno virtualizado sobre un procesador de escritorio de 13.º generación Intel Core i5-13400F (Raptor Lake, 10 núcleos / 16 CPUs lógicas: 6 P-Cores con Hyper-Threading y 4 E-Cores). La prueba corrió sobre un sistema anfitrión Windows 11 utilizando una máquina virtual Linux (WSL2 con hipervisor Hyper-V), asignando acceso directo a la memoria RAM física.

Tabla 9: Resultados del experimento en entorno virtualizado (Intel Core i5-13400F).

N	Algoritmo / Mecanismo	Hilos (p)	Tiempo (ms)	Desv. Est. (ms)	CV (%)
96	Posix FxT	1	0.38	0.04	10.5 %
96	Posix FxT	12	0.12	0.02	16.6 %
96	Fork FxC	16	1.05	0.18	17.1 %
240	Posix FxT	12	1.25	0.09	7.2 %
480	Posix FxT	12	8.42	0.31	3.7 %
720	Posix FxT	12	22.15	0.84	3.8 %
960	Posix FxT	12	51.30	1.95	3.8 %
1440	Posix FxT	12	168.40	4.20	2.5 %
1920	Posix FxT	12	398.10	8.60	2.2 %
2400	Posix FxT	12	752.30	14.10	1.9 %
2400	Posix FxC	1	16 850.00	210.00	1.2 %

Análisis del entorno virtualizado.

- **Desempeño absoluto:** La configuración `PosixFxT` con 12 hilos alcanzó un tiempo de ejecución de 0.75 s para $N = 2400$, superando tanto al Ryzen 5 5600H (1.08 s) como al HP Victus i5-12450H (0.85 s). Esto representa un rendimiento máximo aproximado de 36.8 GFLOPS.
- **Costo de la virtualización en `fork`:** En matrices pequeñas ($N = 96$), la creación de procesos mediante `fork` dentro de la máquina virtual registra tiempos de hasta 1.05 ms (con un CV del 17.1 %), reflejando la penalización adicional introducida por el hipervisor al interceptar y gestionar llamadas al sistema de gestión de memoria.

- **Escalabilidad en arquitectura híbrida:** Al igual que en el HP Victus, la mejor configuración de hilos se obtiene con 12 hilos (coincidiendo con la asignación óptima entre P-Cores y E-Cores), evidenciando que sobrecargar hasta 16 hilos en WSL2 genera ligera contención por cambio de contexto en el hipervisor.

8. Limitaciones

- Las laptops tenían entorno gráfico activo (con aplicaciones cerradas); el servidor no. Es una diferencia de condiciones entre las máquinas.
- Los compiladores no son iguales (gcc 14.2, gcc 12.2 y gcc 13.3). Todos usan `-O3`, pero el código generado puede diferir.
- Con $N = 96$ los tiempos son tan cortos que el ruido domina; esos datos sirven para ver el costo de crear hilos y procesos, no para comparar algoritmos.
- El HP Victus tiene un procesador híbrido y entorno gráfico activo: con 12 y 16 hilos el resultado depende de cómo el planificador reparta los hilos entre núcleos P y E, y los datos de esta máquina son estimados.

9. Conclusiones y recomendaciones

9.1. Conclusiones

- **El patrón de acceso a memoria es el factor más determinante.** FxT es igual o mejor que FxC en todos los casos, y la ventaja crece cuanto menos caché tiene la máquina (hasta 4.2×).
- **Paralelizar sirve hasta el número de núcleos**, o de CPUs lógicas si el algoritmo deja tiempos muertos, como FxT. Pasar de ahí siempre empeoró el rendimiento.
- **En una máquina con memoria lenta, más hilos puede ser peor.** Hay que medir y no asumir.
- **Para trabajos largos, hilos y procesos son equivalentes**; para trabajos cortos, los hilos son preferibles porque su creación es más barata.
- **Ciertos tamaños** (múltiplos de potencias de 2 grandes) provocan conflictos de caché que el análisis $O(N^3)$ no predice.
- El Ryzen es unas 2.5 veces más rápido por núcleo que el Core 2 Duo y hasta 15 veces más rápido aprovechando todo su paralelismo (mejor configuración contra mejor configuración).
- El HP Victus es la máquina más rápida (0.85 s con $N = 2400$, 33 GFLOPS de máximo), pero su arquitectura híbrida hace que el escalado de 8 a 12 hilos (los núcleos E) rinda menos que el del Ryzen: 8.60× contra 9.58×.

9.2. Recomendaciones

- Usar el algoritmo FxT (o transponer B) siempre que N no quepa en caché.
- Fijar el número de hilos al número de núcleos físicos, o de CPUs lógicas si hay SMT y el cálculo tiene dependencias que dejan huecos; no superarlo.
- No paralelizar problemas pequeños: con $N = 96$ el costo de crear hilos o procesos supera la ganancia.
- Evitar tamaños de matriz múltiplos de potencias de 2 grandes, o agregar relleno (*padding*) a las filas.
- Al comparar sistemas, medir siempre con muchas repeticiones, controlar frecuencia y servicios, y reportar la variabilidad junto con el promedio.

Referencias

[1] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2019.

[2] R. Jain, *The Art of Computer Systems Performance Analysis*. Wiley, 1991.

[3] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conference*, 1967, pp. 483–485.

[4] J. L. Gustafson, "Reevaluating Amdahl's law," *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988.

[5] U. Drepper, "What every programmer should know about memory," Red Hat, Inc., 2007.

[6] Intel Corporation, *Intel 64 and IA-32 Architectures Optimization Reference Manual*, y especificaciones del procesador Intel Core i5-12450H. https://ark.intel.com/

A. Comandos para repetir el experimento

Pasos para correr la batería en una máquina nueva. Con el mismo `lanzador.pl` los datos son comparables.

```
tar -xvf evalRend2630.tar && cd evalRend2630
sudo apt install build-essential # si no hay gcc/make
mkdir -p bin Soluciones && make All && ls bin
./bin/mxmPosixFxC 4 2 # verificar que multiplica bien
./bin/mxmPosixFxC 1440 1 # calibrar: tiempo del caso grande (us)
lscpu > hw.txt; lscpu -C >> hw.txt; free -h >> hw.txt; uname -a >> hw.txt
echo performance | sudo tee /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor
rm -f Soluciones/* && nohup ./lanzador.pl > lanzador.log 2>&1 &
pgrep -a mxm # ver que prueba va corriendo
ls Soluciones | wc -l # al terminar: 224
wc -l Soluciones/*.dat | tail -1 # al terminar: 6720 en total
```

Se recomienda conectar el equipo a la corriente, usar el perfil de energía de rendimiento y una base de refrigeración para limitar el estrangulamiento térmico.